

TRABALHO PARA A DISCIPLINA DE TÉCNICAS DE PROGRAMAÇÃO – TURMAS S71 e S73 - DOS CURSOS DE EC & BSI DA UTFPR Curitiba: *MILITARY ZONE++*

Fernando Serrilho Prados, Pedro Paiva Vianna
fernandoprados@alunos.utfpr.edu.br , p.vianna2006@alunos.utfpr.edu.br

Disciplina: **Técnicas de Programação – CSE20 / S71** – Prof. Dr. Jean M. Simão
Departamento Acadêmico de Informática – DAINF - Campus de Curitiba
Curso Bacharelado em: Engenharia da Computação (EC) / Sistemas de Informação (BSI)
Universidade Tecnológica Federal do Paraná - UTFPR
Avenida Sete de Setembro, 3165 - Curitiba/PR, Brasil - CEP 80230-901

Resumo:

A disciplina de Técnicas de Programação exige o desenvolvimento de um *software* de plataforma, no formato de um jogo, para fins de aprendizado de técnicas de engenharia de *software*, particularmente de programação orientada a objetos em C++. Para tal, neste trabalho, escolheu-se o jogo Military Zone++, no qual o jogador enfrenta inimigos em um dado cenário. O jogo tem duas fases que se diferenciam por dificuldades para o jogador. Para o desenvolvimento do jogo foram considerados os requisitos textualmente propostos e elaborada modelagem (análise e projeto) via Diagrama de Classes em Linguagem de Modelagem Unificada (*Unified Modeling Language - UML*) usando como base um diagrama assaz genérico e prévio proposto. Subsequentemente, em linguagem de programação C++ 2003, realizou-se o desenvolvimento que contemplou os conceitos usuais de Orientação a Objetos como Classe, Objeto e Relacionamento, bem como alguns conceitos ditos avançados como Classe Abstrata, Polimorfismo, Gabaritos, Persistências de Objetos por Arquivos, Sobrecarga de Operadores e Biblioteca Padrão de Gabaritos (*Standard Template Library - STL*). Depois da implementação, os testes e uso do jogo feitos pelos próprios desenvolvedores demonstraram sua funcionalidade conforme os requisitos e a modelagem elaborada. Por fim, salienta-se que o desenvolvimento em questão permitiu cumprir o objetivo de aprendizado visado.

Expressões-chave: Artigo-Relatório para o Trabalho em Técnicas de Programação, Trabalho Acadêmico Voltado a Implementação em C++ 2003, Ciclo de engenharia de Software.

INTRODUÇÃO

O trabalho em questão foi feito para a matéria técnicas de programação 2026/1 na UTFPR-CT em Curitiba, lecionada pelo Dr. Jean M. Simao, o qual foi usado como método de avaliação para compor a nota ao fim do semestre letivo, visando usar em um projeto os conhecimentos adquiridos na matéria até o momento. Além disso, também foi requerido um diagrama UML do projeto, para assim, adquirir conceitos básicos de engenharia de software na criação de diagramas deste tipo. Os objetivos principais são aplicar os conhecimentos da orientação de objetos em C++, desenvolver um diagrama em UML, como seguir a lista de requerimentos de um projeto o qual nos foi dado e se acostumar com o trabalho em equipe.

O projeto em questão é o desenvolvimento em dupla de um jogo 2d em plataforma aplicando os conceitos de orientação de objetos, o projeto tem por objetivo aplicar conceitos de programação via orientação a objetos em C++. Com isso, é requerido o uso de conceitos como classes abstratas, polimorfismo, persistência de objetos por arquivo e outros, usando a biblioteca gráfica do SFML.

Para desenvolver o projeto, foi adotado o ciclo de Engenharia de Software, de suma importância para tal. Antes do código, foi analisado os requisitos exigidos para o projeto, como seguir o que era apresentado tanto no diagrama UML disponibilizado pelo professor quanto nas tabelas de requisitos, com foco nos princípios da orientação a objetos, polimorfismo e coesão dentro do código. Durante a modelagem do UML, foi importante na

organização e uma compreensão visual de como as classes relacionam-se entre si, seja na herança e polimorfismo de classes, com nas relações de associação, composição ou agregação.

O artigo foi separado de forma a explicar o jogo de forma simples, cobrindo todas as etapas envolvidas nele, seguido de um foco mais técnico no desenvolvimento do jogo, focando nos conceitos utilizados e metodologia usada na construção do projeto junto do diagrama, além da tabela dos conceitos elementares usados, seguido da conclusão de toda o jornada deste trabalho e de da distribuição de tarefas entre os integrantes envolvidos. Por fim, temos nossos agradecimentos e as referências usadas durante o jogo.

EXPLICAÇÃO DO JOGO EM SI

Para o jogo de plataforma, foi escolhido um tema militar, com as fases tendo a câmera estática, composta por obstáculos, inimigos e um ou dois jogadores simultâneos durante a fase.

O jogo inicia no Menu inicial, contendo uma imagem de BackGround [E] e opções que o usuário possa escolher, onde vê-se na Figura 1.

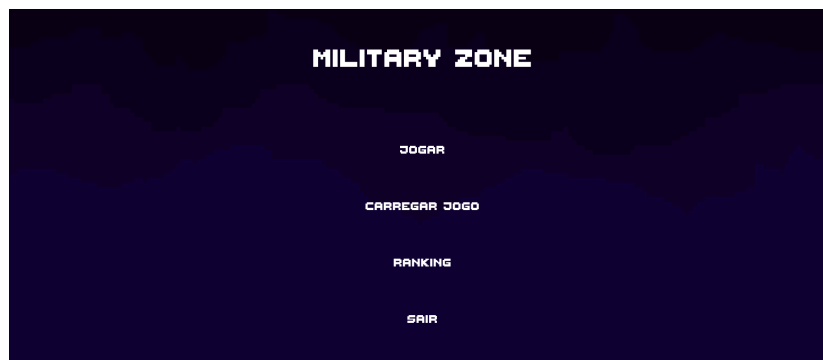


Figura 1. Menu de Início do jogo.

O usuário pode então escolher se quer ver o ranking da pontuação de jogadores, podendo escolher entre ver o ranking da fase um ou da fase dois, como na Figura 2.



Figura 2. Ranking da Fase Um e Fase Dois respectivamente.

Na opção de carregar , caso haja um jogo salvo na memória, o usuário volta nele a partir do momento em que o salvamento foi realizado, continuando a fase daquele instante.

Por fim, ao clicar em jogar, o usuário escolhe entre um dois jogadores, em seguida, escolhe o nome de cada jogador, e por fim, escolhe entre jogar a fase um ou a fase dois, fazendo assim iniciar alguma das fases.

Ao iniciar a fase um, o jogo irá carregar uma imagem de fundo[A], um chão[F], dois tipos de obstáculos, a plataforma[G] e o arbusto[C] e dois tipos de inimigos drones[H] e soldados[I]., como visto na Figura 6.

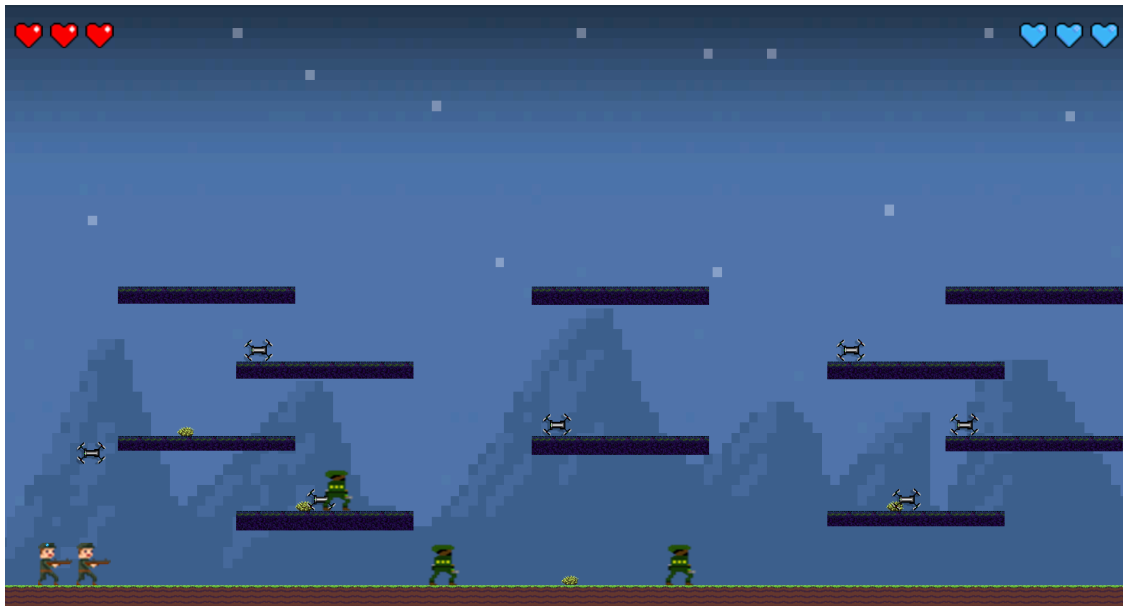


Figura 6. Fase um do Jogo.

Na segunda fase, temos uma nova imagem para o fundo[D] e chão[N], dois tipos de obstáculos, a plataforma e a mina terrestre[M] e dois tipos de inimigos, o drone e o tanque[K], como visto na Figura 7.



Figura 7. Fase dois do jogo.

Durante a execução de qualquer uma das fases, também há um menu de pausa nele Figura 8, onde você pode escolher entre continuar o jogo, sair dele, salvar o seu progresso atual ou voltar para o menu do jogo.



Figura 8. Menu de pausa do jogo.

Também há um menu de derrota caso o personagem perca todos seus pontos de vida, como o da Figura 9.



Figura 9. Menu de Derrota do jogo.

Por fim, também há, no momento de conclusão de uma fase, um menu onde você pode prosseguir para a próxima fase, sair, voltar ao menu ou reiniciar a fase, na Figura 10.

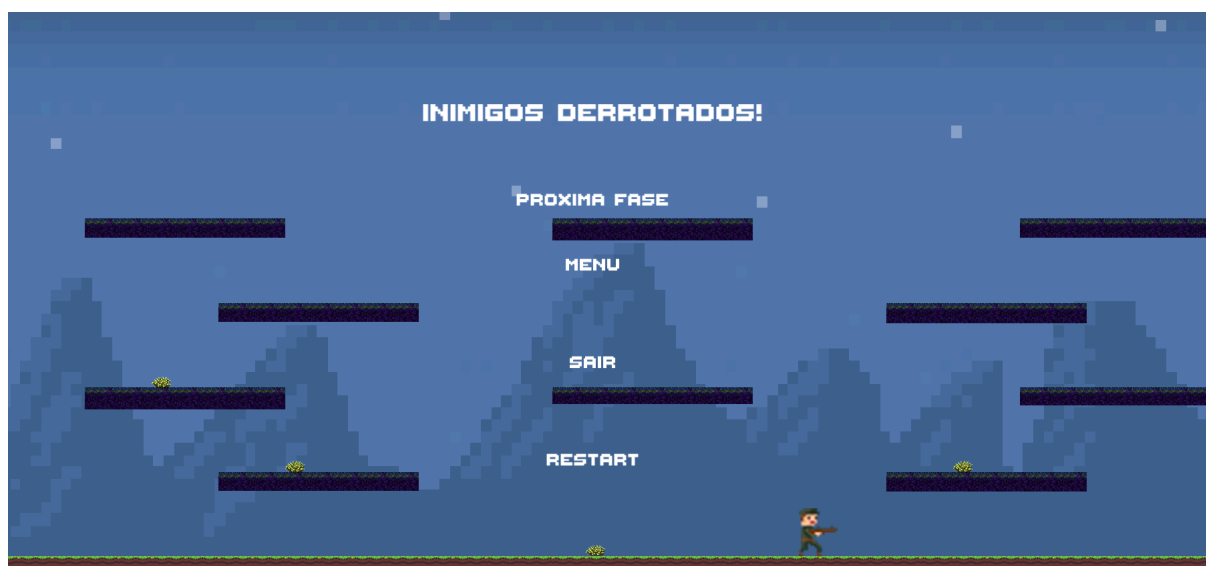


Figura 10. Menu de Conclusão de Fase.

DESENVOLVIMENTO DO JOGO

Pensando numa forma de construir um código estável, com uma lógica clara e que facilita a manutenção se necessária, o uso de um ciclo de engenharia de software (previamente comentada na introdução) se fez muito útil para tal, e seguindo isso, a primeira etapa seria entender e compreender os requisitos pedidos, para que dessa forma, a construção de toda a estrutura não só do código, mas como também da organização do projeto em um geral. Para realizar a clara compreensão destes requisitos, a Tabela 1 providenciada pelo professor, foi de suma importância, o qual os tópicos exigidos foram seguidos à risca, a fim de priorizar este requisitos funcionais, tratados com prioridade máxima ao decorrer do projeto.

Tabela 1. Lista de Requisitos do Jogo de Plataforma e exemplos de Situações.

N.	Requisitos Funcionais	Situação	Implementação
1	Apresentar graficamente menu de opções aos usuários do Jogo, no qual pode se escolher fases, escolher ver colocação (<i>ranking</i>) de jogadores e escolher demais opções pertinentes (previstas nos demais requisitos).	REALIZADO	O menu possibilita acessar as fases pertinentes, sair do jogo e visualizar o ranking geral de pontuação por fase.
2	Permitir um ou dois jogadores com representação gráfica aos usuários do Jogo, sendo que no último caso é para que os dois joguem de maneira concomitante.	REALIZADO	Dois jogadores implementados, controlados de maneira concomitante.
3	Disponibilizar ao menos duas fases <u>distintas</u> , com chão, que podem ser jogadas sequencialmente ou selecionadas, via menu, nas quais jogadores tentam neutralizar inimigos por meio de algum artifício e vice-versa.	REALIZADO	Duas fases dadas via menu e sequencial, com os jogadores tendo uma forma de neutralizar, assim como os inimigos de neutralizar os jogadores.
4	Ter pelo menos três tipos distintos de inimigos, cada qual com sua representação gráfica, sendo que ao menos um deles deve poder lançar projétil	REALIZADO	Um drone, que segue o jogador no ar, um soldado que se move de

	contra o(s) jogador(es) e um dos inimigos deve ser um 'chefão'.		um lado para o outro, e um tanque que atira projéteis.
5	Ter a cada fase ao menos dois tipos de inimigos (<u>um deles exclusivo nela</u>) com número aleatório de instâncias, podendo ser várias instâncias (<u>definindo um máximo – e.g. dez</u>) e sendo pelo menos 3 instâncias <u>para cada tipo que estiver na fase</u> .	REALIZADO	Drone e soldado na fase um, e drone e tanque na fase dois, com instâncias aleatórios com mínimo de três
6	Ter três tipos de obstáculos, cada qual com sua representação gráfica, sendo que ao menos um causa dano em jogador se eles colidirem.	REALIZADO	Uma plataforma, um arbusto, que aplica lentidão ao jogador, e uma mina terrestre, que aplica dano ao jogador.
7	Ter em cada fase ao menos dois tipos de obstáculos (<u>um deles exclusivo nela</u>) com número aleatório (<u>definindo um máximo</u>) de instâncias (<i>i.e.</i> , objetos), sendo pelo menos 3 instâncias por tipo.	REALIZADO	Plataforma e arbusto na fase um, e plataforma e mina terrestre na fase dois, com a menos 3 instâncias cada.
8	Ter em cada fase um cenário de jogo constituído por obstáculos, sendo que parte deles devem ser plataformas ou similares, sobre as quais pode haver inimigos e podem subir jogadores. Em cada fase, só pode ter um tipo coincidente de inimigo e um tipo coincidente de obstáculo (que é a plataforma) em relação às demais fases.	REALIZADO	Aplicado da maneira requerida em ambas as fases.
9	Gerenciar colisões entre jogador para com os inimigos e seu conjunto de projéteis, bem como entre jogador para com os obstáculos. Ainda, todos eles devem sofrer o efeito de alguma 'gravidade' no âmbito deste jogo de plataforma vertical e 2D.	REALIZADO	Gerenciador de Colisões gerencia todos estes eventos requeridos. Aplicada a gravidade a todos os elementos pertinentes.
10	Permitir: (1) salvar nome do usuário, manter/salvar pontuação (incrementada via neutralização de inimigos) do jogador controlado pelo usuário e gerar lista de pontuação (<i>ranking</i>). E (2) Pausar e Salvar/Recuperar Jogada.	REALIZADO	Ao fim das fases, o jogo verifica se o jogador é salvo no ranking, assim como o jogo pode ser salvo tanto pelo menu do jogo, quanto caso o jogo seja fechado, podendo ser carregado pelo menu inicial.
Total de requisitos funcionais apropriadamente realizados. (Cada tópico realizado efetivamente vale 10%)			100%
Os requisitos dependem em algo uns dos outros, na chamada interdependência de requisitos.			

Após isso, foi usado como base o diagrama UML disponibilizado pelo professor, respeitando e seguindo uma estrutura muito similar ao que foi passado, especialmente na criação das classes e forma como eles se relacionavam entre elas.

Com isso pode destacar-se de começo os Gerenciadores, composto pelo Gerenciador Gráfico, responsável por toda a parte gráfica que atua para abrir a janela do jogo e desenhar os textos e sprites, seguido do Gerenciador de Colisões, ele que possui ferramentas que conhecem todos os inimigos, obstáculos e projéteis no jogo, que atua na verificação da colisão dessas entidades e o que acontece na interação entre eles.

A seguir temos uma coleção de uma lista, com duas classes aninhadas, uma para receber o tipo do elemento, e um iterador próprio, coleção esta que é classe mãe para a Lista Entidades, responsável por gerir todas as entidades presentes no jogo, gerindo a execução dos métodos virtuais das entidades.

Vê-se também a classe Jogo, a principal dentro do jogo, responsável pela criação das fases e dos jogadores, onde durante a execução de fases confere sempre a situação da fase, como se os jogadores permanecem vivos ou se todos os inimigos foram neutralizados, além de administrar a execução da fase ativa.

Destaca-se também o menu, responsável por administrar todos os estados de Menu, utilizando de formas de persistência de objetos para armazenar dados referentes ao ranking em arquivos de tipo.txt.

Em seguida, temos o conjunto de Entidades, que engloba todas as classes que de alguma forma possuem uma representação gráfica no jogo e uma utilidade, onde se vê de forma muito presente os conceitos de polimorfismo, abstração de atributos e métodos, além de métodos virtuais e virtuais puros muito presentes, como os inimigos, objetos de cenário, jogador e obstáculos.

Por fim no conjunto de Fases, engloba ambas as fases do jogo, onde criam-se os elementos que compõem cada fase, como obstáculos, inimigos e o cenário, além de possuírem uma forma de acessar o Gerenciador de Colisões, e uma lista com todas as entidades presentes na fase.

Estes e outros detalhes podem ser melhor vistos na Figura 11 que contém o diagrama UML do projeto.

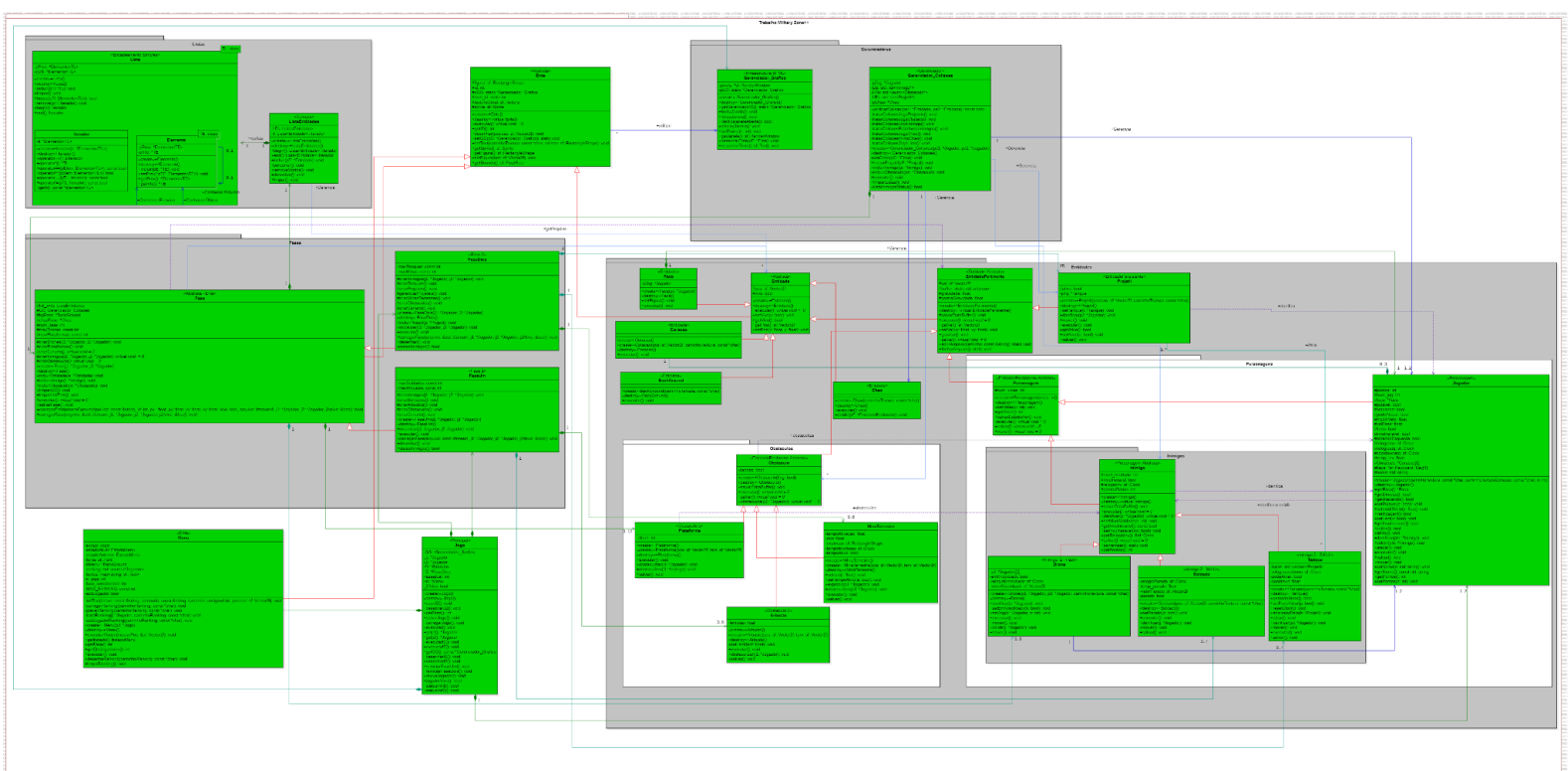


Figura 11. Diagrama de Classes de base em UML

TABELA DE CONCEITOS UTILIZADOS E NÃO UTILIZADOS

Outro dos requisitos pedidos, seria a implementação de conceitos elementares, servindo como uma forma de aplicar conceitos desde mais simples a mais avançados como o uso de Threads, RAD e Padrões de Projeto. De antemão, nem todos foram implementados, seja por falta de tempo ou priorização de outros conceitos.

Isso tudo pode ser visto na Tabela 2 a seguir.

Tabela 2. Lista de Conceitos Utilizados e Não Utilizados no Trabalho.

N.	Conceitos	Uso	O quê / Onde & Justificativa sucinta em uma frase
1	Elementares:		
1.1 &	- Classes, objetos. & - Atributos (privados), variáveis e constantes. - Métodos (com e sem retorno).	Sim	- Todos .h e .cpp - Classes, Objetos, Atributos e Métodos foram utilizados porque são conceitos elementares na orientação a objetos.
1.2 &	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). - Construtores (sem/com parâmetros) e destrutores	Sim	- Utilizada na classe Iterador da Lista. - A constância pertinente evita mudanças equivocadas, construtores são mandatórios para inicializar atributos e destrutores pertinentes para finalizações como desalocações.
1.3	- Classe Principal.	Sim	- Main.cpp & Jogo.h/.cpp - Uma classe Principal é mais 'purista' em termos de OO.
1.4	- Divisão em .h e .cpp.	Sim	- Em todas as classes exceto na estrutura de dados Lista por ser template - Permite organizar as classes e afins que compõem o sistema.
2	Relações de:		
2.1	- Associação direcional. & - Associação bidirecional.	Sim	- Associação direcional usada para a classe Gerenciador de Colisões com o Jogador e Inimigos, mantendo eles independentes pois não precisam conhecer o Ger. Colisões. - Associação bidirecional usada para as classes Tanque e Bala, necessária para sincronizar as ações e posições das duas classes.
2.2 &	- Agregação via associação. - Agregação propriamente dita.	Sim	- Agregação via associação utilizada na FaseUm, que instância o Chão e o armazena na lista de entidades, permitindo o gerenciamento do cenário de forma desacoplada.
2.3 &	- Herança elementar. - Herança em vários níveis.	Sim	- Herança elementar acontece em diversos pontos do código, como na Fase Um herdando a Fase - Herança em vários níveis acontece por exemplo nos Drones e Soldados que herdam do Inimigo que herda do personagem que herda da entidade que por sua vez herda do Ente.
2.4	- Herança múltipla pertinente.	Não	- Não implementada
3	Ponteiros, generalizações e exceções		
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional	Sim	- Utilizado na Classe Jogador para a Classe Faca, permitindo a Faca a descobrir a posição do jogador.
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Sim	- Utilizado na Construtora e Destrutora do Jogador para a Instância da sua Faca, visando o gerenciamento de memória pois não é preciso a faca continuar existindo se o jogador foi morto. Além de ser implementada na classe template Lista. -
3.3	- Gabaritos/Templates criada/adaptados pelos autores para Listas.	Sim	- Implementado template na classe Lista para a lista de entidades da fase. A fim de seguir o diagrama UML do modelo.

3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Sim	- Implementada no danificar da classe Jogador, para verificar se o inimigo recebido é nulo ou não..
4	Sobrecarga de:		
4.1	- Construtoras e Métodos.	Sim	-Utilizada no método Obstaculizar dos obstáculos, vantajoso para tratar diferentemente para Jogadores e Inimigos.
4.2	- Operadores (2 tipos de operadores pelo menos).	Sim	Foram utilizados na classe Iterador dentro da Lista, especificamente os <code>operator++</code> , <code>operator*</code> , <code>operator==</code> e <code>operator!=</code> .
---	Persistência de Objetos (via arquivo de texto ou binário)		
4.3	- Persistência de Objetos.	Sim	- Implementada em todas as funções derivadas de EntidadePertinente. - Chamadas quando se salva o progresso em uma fase .
4.4	- Persistência de Relacionamento de Objetos.	Sim	- Implementada na relação entre projétil e tanque, com ambos se conhecendo, servindo para guardar de qual tanque pertence o projétil.
5	Virtualidade:		
5.1	- Métodos Virtuais Usuais.	Sim	-Utilizados Para o <code>setPos</code> e <code>setVel</code> de entidades, pois foi necessário sobrescrevê-los na Classe Jogador.
5.2	- Polimorfismo extensivamente à luz do padrão arquitetural (i.e., diagrama de classes assaz genérico) dado.	Sim	-Utilizado no método percorrer da Lista Entidades sendo utilizado na classe FaseUm, onde chama o executar de cada entidade, importante pois cada entidade necessita de um executar diferente.
5.3	- Métodos Virtuais Puros / Classes Abstratas à luz do padrão arquitetural.	Sim	-Utilizados em todas as Classes bases, como na função <code>executar()</code> , e <code>mover()</code> , importante para que todas as classes derivadas delas tenham as funções necessárias.
5.4	- Coesão/Desacoplamento efetiva e intensa inclusive com o apoio de mais de 5 padrões de projeto.	Não	-Não implementada
6	Organizadores e Estáticos		
6.1	- Espaço de Nomes (<i>Namespace</i>) criada pelos autores.	Sim	-Implementados todos os namespaces que estavam no UML do modelo.
6.2	- Classes aninhadas (<i>Nested</i>) criada ou adaptadas pelos autores.	Sim	-Utilizada na classe Lista, que possui as classes Iterador e Elemento como aninhadas.
6.3	- Atributos estáticos e métodos estáticos.	Sim	-Utilizado na Classe Gerenciador de Colisões , a fim de utilizar do padrão Singleton, como mostrado na reunião do PETECO.
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Sim	-Utilizada amplamente na classe Iterador da Lista, a fim de impedir que os métodos <code>operator</code> de comparação alterem algo.
7	Standard Template Library (STL) e String OO		
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da <i>STL</i> (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Sim	-String utilizada na classe menu para gerenciar os textos das opções. -Vector utilizados para as listas de inimigos e obstáculos do gerenciado de colisões, a fim de seguir o diagrama UML do modelo.
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Sim	-Mapa Implementado na Classe Menu para controle dos textos impressos em tela.
---	Programação concorrente		
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não	-Não implementado

7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não	-Não implementado
8	Biblioteca Gráfica / Visual		
8.1 &	- Funcionalidades Elementares. - Funcionalidades Avançadas como: tratamento de colisões, duplo <i>buffer</i> ou outros.	Sim	- Funcionalidades elementares ao longo de todo o código, assim como o tratamento de colisões especialmente presente no gerenciador de Colisões
8.2 OU	- Programação orientada e evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. - <i>RAD – Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não	-Não implementado
---	Interdisciplinaridades via utilização de Conceitos de Matemática Contínua e/ou Física.		
8.3	- Ensino Médio Efetivamente.	Sim	- Gravidade utilizando a fórmula $y = y_0 + v_0t + at^2/2$
8.4	- Ensino Superior Efetivamente.	Não	-Não implementado
9	Engenharia de Software		
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Sim	-Utilizado Google Docs e o modelo disponibilizado pelo simão para controlar os requisitos, além de ligações no Discord frequentes para discussão sobre tal implementação do requisito.
9.2	- Diagrama de Classes em <i>UML</i> .	Sim	-Utilizado o StarUML, com base no diagrama UML disponibilizado no modelo do professor.
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , <i>i.e.</i> , mais de 5 padrões.	Não	-Apenas o singleton no gerenciador de colisões.
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Não	-Não foi realizado.
10	Execução de Projeto		
10.1 &	- Controle de versão de modelos e códigos automatizado (via github). - Uso de alguma forma de cópia de segurança (<i>i.e.</i> , <i>backup</i>).	Sim	-Controle de versão via github: https://github.com/FernandoSerrilho/Jogo_Tech_Prog
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	- 2 Reuniões foram realizadas junto do professor da matéria com a presença de ambos os integrantes. Primeira Reunião 09/06/26 às 09:00 horas. Segunda Reunião 16/09/26 às 09:00 horas.
10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA DO TRABALHO]	Sim	- Pedro Paiva: Projeto Simas 15/05 Introdução & Discussão Projeto Simas 29/05 Gerenciador Gráfico & Gerenciador de Colisões. - Fernando Prados: Projeto Simas 15/05 Introdução & Discussão Projeto Simas 29/05 Gerenciador Gráfico & Gerenciador de Colisões.

10.4 &	- Escrita do trabalho e feitura da apresentação - Revisão do trabalho escrito de outra equipe e vice-versa.	<i>Sim</i>	- Relatório escrito por Pedro Paiva, enquanto a apresentação foi realizada por Fernando Prados, com revisão posteriormente devidamente feita. - A equipe de Vinicius Guimarães e Luiza Comiran revisaram nosso trabalho escrito e vice-versa. - A equipe de Hélio Repczuk e Raquel de Toledo revisaram nosso trabalho e vice-versa.
Total de conceitos apropriadamente utilizados.			75%

DISCUSSÃO E CONCLUSÕES

Com a finalização do projeto, pode-se destacar que os conceitos vistos e aprendidos sobre a programação via Orientação a Objetos foram devidamente aplicados e usados, de forma a desenvolver para ambos, um melhor domínio acerca do tema abordado, apesar de não ter sido feito todos os elementos da tabela de conceitos elementares.

Vale dizer também que com o ciclo de Engenharia de Software, ficou claro como ele deve ser seguido e como realizar um diagrama do tipo UML, algo que na indústria, pode ser requerido de ser feito.

Também destaca-se as reuniões feitas com o professor e integrantes do Projeto PETECO que, além de auxiliar em dúvidas ou problemas ao longo do desenvolvimento, foram de extrema importância para dar uma direção ou redirecionar o rumo que o projeto estava tomando, dando sugestões de onde começar, como fazer, onde focar e a distribuir melhor as tarefas.

O projeto também evoluiu as habilidades dos autores acerca do trabalho em equipe a distância, aprendendo a usar as ferramentas disponíveis, com destaque ao GitHub, ferramenta que ambos possuíam pouca experiência, mas que no final, possuem agora familiaridade com a ferramenta.

DIVISÃO DO TRABALHO

A divisão de trabalho do projeto foi feito de forma que ambos pudessem participar da forma mais igual, sem com que um ficasse mais sobrecarregado que o outro, sendo feito num formato que se aproxima de 50% tendo sido feito em conjunto, enquanto a outra metade foi dividida entre os integrantes, ficando 25% para cada, de tal forma que cada parte feito individualmente por cada um dos autores, foi sendo revisada pelo outro a medida do desenvolvimento do projeto.

A distribuição destas tarefas pode ser vista na Tabela 3 logo a seguir.

Tabela 3. Lista de Atividades e Responsáveis.

N.	Atividades	Responsáveis
1	Elementares: AMBOS	
1.1	- Classes, objetos. & - Atributos (privados), variáveis e constantes. & - Métodos (com e sem retorno).	Fernando e Pedro
1.2	- Métodos (com retorno <i>const</i> e parâmetro <i>const</i>). & - Construtores (sem/com parâmetros) e destrutores	Fernando e Pedro
1.3	- Classe Principal.	Pedro

1.4	- Divisão em .h e .cpp.	Fernando e Pedro
2	Relações de: Mais Fernando	
2.1	- Associação direcional. & - Associação bidirecional.	Fernando
2.2	- Agregação via associação. & - Agregação propriamente dita.	Parcialmente implementada
2.3	- Herança elementar. & - Herança em vários níveis.	Fernando e Pedro
2.4	- Herança múltipla.	Não implementada
3	Ponteiros, generalizações e exceções: Ambos	
3.1	- Operador <i>this</i> para fins de relacionamento bidirecional.	Fernando
3.2	- Alocação de memória (<i>new</i> & <i>delete</i>).	Fernando e Pedro
3.3	- Gabaritos/ <i>Templates</i> criada/adaptados pelos autores para Listas.	Pedro
3.4	- Uso de Tratamento de Exceções (<i>try catch</i>).	Pedro
4	Sobrecarga de: Mais Pedro	
4.1	- Construtoras e Métodos.	Fernando e Pedro
4.2	- Operadores (2 tipos de operadores pelo menos)	Pedro
---	Persistência de Objetos (via arquivo de texto ou binário): Mais Fernando	
4.3	- Persistência de Objetos.	Fernando e Pedro.
4.4	- Persistência de Relacionamento de Objetos.	Fernando.
5	Virtualidade: .. Ambos	
5.1	- Métodos Virtuais Usuais.	Fernando
5.2	- Polimorfismo.	Pedro
5.3	- Métodos Virtuais Puros / Classes Abstratas.	Fernando e Pedro
5.4	- Coesão/Desacoplamento efetiva e intensa com o apoio de padrões de projeto (mais de 5 padrões).	Não implementada
6	Organizadores e Estáticos: .Mais Pedro	
6.1	- Espaço de Nomes (<i>Namespace</i>) criada pelos autores.	Fernando e Pedro
6.2	- Classes aninhadas (<i>Nested</i>) criada pelos autores.	Pedro
6.3	- Atributos estáticos e métodos estáticos.	Pedro
6.4	- Uso extensivo de constante (<i>const</i>) parâmetro, retorno, método...	Fernando e Pedro
7	Standard Template Library (STL) e String OO: .. Mais Pedro	
7.1	- A classe Pré-definida <i>String</i> ou equivalente. & - <i>Vector</i> e/ou <i>List</i> da <i>STL</i> (p/ objetos ou ponteiros de objetos de classes definidos pelos autores)	Fernando e Pedro
7.2	- Pilha, Fila, Bifila, Fila de Prioridade, Conjunto, Multi-Conjunto, Mapa OU Multi-Mapa.	Pedro
---	Programação concorrente: ...	
7.3	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos, utilizando Posix, C-Run-Time OU Win32API ou afins.	Não implementada
7.4	- <i>Threads</i> (Linhas de Execução) no âmbito da Orientação a Objetos com uso de Mutex, Semáforos, OU Troca de mensagens.	Não implementada
8	Biblioteca Gráfica / Visual: Ambos	
8.1	- Funcionalidades Elementares. & - Funcionalidades Avançadas como: tratamento de colisões e duplo <i>buffer</i>	Pedro e Fernando
8.2	- Programação orientada a evento efetiva (com gerenciador apropriado de eventos inclusive, via padrão de projeto <i>Observer</i>) em algum ambiente gráfico. OU - <i>RAD</i> – <i>Rapid Application Development</i> (Objetos gráficos como formulários, botões etc).	Não implementada
---	Interdisciplinaridades via uso de Conceitos de Matemática Contínua e/ou Física: .Mais Fernando	
8.3	- Ensino Médio Efetivamente.	Fernando
8.4	- Ensino Superior Efetivamente.	Não implementada
9	Engenharia de Software: Ambos	
9.1	- Compreensão, melhoria e rastreabilidade de cumprimento de requisitos.	Fernando e Pedro
9.2	- Diagrama de Classes em <i>UML</i> .	Fernando e Pedro
9.3	- Uso efetivo e intensivo de padrões de projeto <i>GOF</i> , i.e., + de 5 padrões.	Não Implementada
9.4	- Testes à luz da Tabela de Requisitos e do Diagrama de Classes.	Não Implementada
10	Execução de Projeto: .Ambos	

10.1	- Controle de versão de modelos e códigos automatizado (via github). & - <i>Uso de alguma forma de cópia de segurança (i.e., backup).</i>	Fernando e Pedro
10.2	- Reuniões com o professor para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO A ENTREGA DO TRABALHO]	Fernando e Pedro
10.3	- Reuniões com monitor da disciplina para acompanhamento do andamento do projeto. [ITEM OBRIGATÓRIO PARA A ENTREGA]	Fernando e Pedro
10.4	- Escrita do trabalho e feitura da apresentação & - Revisão do trabalho escrito de outra equipe e vice-versa.	Fernando e Pedro

AGRADECIMENTOS PROFISSIONAIS

Havendo agradecimentos de ordem profissional, como ajuda dos integrantes PETECO, aos colegas Vinícius Guimarães, Luiza Comiran, Hélio Repczuk e Raquel de Toledo por revisarem nosso trabalho, e ao professor Dr. Jean M. Simão.

REFERÊNCIAS UTILIZADAS NO DESENVOLVIMENTO

- [A] NYKNCK. Background Set. Itchi.io, 2019. Disponível em: <<https://nyknck.itch.io/background-set-pixel-assets>>. Acesso em: 01 jun. 2026.
- [B] HARRINGTON, D. WW2 Soldier. OpenGameArt.org, 2016. Disponível em: <<https://opengameart.org/content/ww2-soldier>>. Acesso em: 30 mai. 2026.
- [C] KARSIORI. Free Pixel Art Bush Pack. Itchi.io, 2016. Disponível em: <<https://karsiori.itch.io/free-pixel-art-bush-pack>>. Acesso em: 14 jun. 2026.
- [D] SAUER2. Starfield Background. OpenGameArt.org, 2011. Disponível em: <<https://opengameart.org/content/starfield-background>>. Acesso em: 14 jun. 2026.
- [E] GAMER247. Sky. Itchi.io, 2019. Disponível em: <<https://gamer247.itch.io/sky>>. Acesso em: 10 jun. 2026.
- [F] IMDANIELL. Dirt Walls and Plataforms. Itchi.io, 2016. Disponível em: <<https://imdaniell.itch.io/dirt-walls-and-platforms>>. Acesso em: 01 jun. 2026.
- [G] THOMASWP. Plataformer Night Tilset. OpenGameArt.org, 2013. Disponível em: <<https://opengameart.org/content/platformer-night-tileset>>. Acesso em: 17 jun. 2026.
- [H] RUOK. Pixel Art Drone. OpenGameArt.org, 2025. Disponível em: <<https://opengameart.org/content/pixel-art-drone>>. Acesso em: 06 jun. 2026.
- [I] RUOK. Green Soldier. OpenGameArt.org, 2016. Disponível em: <<https://opengameart.org/content/green-soldier>>. Acesso em: 11 jun. 2026.
- [J] GRUMPYDIAMOND. PixelWeapons1 OpenGameArt.org, 2016. Disponível em: <<https://opengameart.org/content/pixel-weapons1>>. Acesso em: 11 jun. 2026.
- [K] MRNANNINGS. PixelTank OpenGameArt.org, 2015. Disponível em: <<https://opengameart.org/content/pixel-tank>>. Acesso em: 14 jun. 2026.

- [L] UNREACHED L. Lifebar Itchi.io, 2025. Disponível em: <<https://unreached-lands.itch.io/lifebar-pixelart-sprites-16x16>>. Acesso em: 13 jun. 2026.
- [M] PAIVA P. Mina Terrestre Google Drive, 2026. Disponível em: <https://drive.google.com/file/d/1eCCxQ479SYTTpjwk2PwgGPxOtp-_eV8o/view?usp=drive_link>. Acesso em: 13 jun. 2026.
- [N] PLLGD. Stone Ground, OpenGameArt.org, 2019. Disponível em: <<https://opengameart.org/content/stone-ground>>. Acesso em: 17 jun. 2026.
- [O] BRIGHAMKEYS. Weapon Icons, OpenGameArt.org, 2016. Disponível em: <<https://opengameart.org/content/weapon-icons-1>>. Acesso em: 17 jun. 2026.
- [P] GEGE++. Página inicial. YouTube, 2021. Disponível em: <<https://www.youtube.com/channel/UCUa8BOx2F3hlxgPcpZmnBnQ>>. Acesso em: 27 mai. 2026.
- [Q] DEV FELIPE. Página inicial. YouTube, 2020. Disponível em: <<https://www.youtube.com/@devfelipe8214>>. Acesso em: 02 jun. 2026.